Python Training Program

Module 13: Data Visualization with Seaborn

Instructor Training Material & Post-Training Study Guide

Covers: Seaborn Fundamentals • Distribution Plots • Categorical Plots
Regression Plots • Matrix/Multivariate Plots • Hands-on Lab

13.1 Introduction to Seaborn

Seaborn is a Python data visualization library built on top of Matplotlib. It provides a high-level interface for creating attractive and informative statistical graphics. Seaborn is tightly integrated with Pandas DataFrames, making it extremely convenient for data analysis workflows. It comes with beautiful default themes, intelligent color palettes, and functions designed specifically for statistical exploration.

Seaborn vs Matplotlib — Abstraction Level, Defaults, Integration

Understanding the difference between Seaborn and Matplotlib helps you choose the right tool and combine them effectively.

- **Matplotlib:** Matplotlib is the foundational plotting library in Python. It gives you complete control over every element of a plot, but requires more code for common statistical charts.
- **Seaborn:** Seaborn wraps around Matplotlib and provides high-level functions that produce complex statistical plots with very little code. Seaborn's defaults are much more aesthetically pleasing.
- **Integration:** Integration: Seaborn returns Matplotlib Axes objects. You can always use plt.show(), plt.savefig(), and other Matplotlib functions alongside Seaborn code.

Example — Creating a scatter plot in both libraries:

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

tips = sns.load_dataset('tips')  # Built-in Seaborn dataset

# Matplotlib approach — more verbose
fig, ax = plt.subplots()
ax.scatter(tips['total_bill'], tips['tip'], alpha=0.6)
ax.set_xlabel('Total Bill')
ax.set_ylabel('Tip')
ax.set_title('Total Bill vs Tip (Matplotlib)')
plt.show()

# Seaborn approach — concise with built-in intelligence
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='sex')
plt.title('Total Bill vs Tip (Seaborn)')
plt.show()
```

Note: *Seaborn automatically color-codes by 'sex', adds a legend, and uses a cleaner style — all from one line of code.*

Installing Seaborn

Seaborn is installed using pip or conda. Most data science environments (Anaconda, Google Colab, Jupyter) already include it.

```
# Using pip
pip install seaborn

# Using conda
conda install seaborn
```

```
# Verify installation
import seaborn as sns
print(sns.__version__)  # e.g., 0.13.2

# Standard imports used in this module
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Seaborn Themes and Context — set_theme(), set_context(), set_palette()

Seaborn provides powerful styling tools that control the overall look of your charts. These should typically be called once at the beginning of your script or notebook.

set_theme()

Controls the overall visual style of plots. Available styles: darkgrid, whitegrid, dark, white, ticks.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Apply a theme globally
sns.set_theme(style='whitegrid')  # Clean white background with grid

# Other style options:
sns.set_theme(style='darkgrid')   # Gray background with white grid
sns.set_theme(style='ticks')      # White background with axis ticks
sns.set_theme(style='dark')       # Dark background, no grid
sns.set_theme(style='white')      # Clean white, no grid

# Reset to default Matplotlib style
sns.reset_defaults()
```

set_context()

Controls the scale of plot elements — labels, lines, tick marks. Useful for adapting output to different presentation sizes.

```
# Contexts: paper, notebook (default), talk, poster
sns.set_context('talk')    # Larger fonts — good for presentations
sns.set_context('poster')  # Largest — for conference posters
sns.set_context('paper')   # Smallest — for printed documents
sns.set_context('notebook')# Default — for Jupyter notebooks

# You can also fine-tune with font_scale
sns.set_context('talk', font_scale=1.2)
```

set_palette()

Sets the default color palette for all subsequent plots. Seaborn has many beautiful built-in palettes.

```
# Qualitative palettes — for categorical data
sns.set_palette('Set2')       # Pastel qualitative
sns.set_palette('tab10')      # Matplotlib default categorical
sns.set_palette('husl')       # Evenly spaced hue colors
```

```
# Sequential palettes — for numeric data
sns.set_palette('Blues')      # Shades of blue
sns.set_palette('YlOrRd')     # Yellow to Red

# Diverging palettes — for data centered around a midpoint
sns.set_palette('RdBu')       # Red to Blue

# View a palette
sns.color_palette('Set2', as_cmap=False)
sns.palplot(sns.color_palette('husl', 8))  # Display 8 colors
plt.show()
```

Tip: Use qualitative palettes for categorical data (gender, city), sequential for ordered numeric data (age, income), and diverging for data with a meaningful center (temperature anomalies, correlation coefficients).

The Figure-level vs Axes-level Function Distinction

This is one of the most important concepts in Seaborn. Every plot function in Seaborn falls into one of two categories, and understanding the difference will save you a lot of confusion.

- **Axes-level functions:** Axes-level functions: Draw on a specific Matplotlib Axes object. They return an Axes object and can be embedded into multi-plot figures created with plt.subplots(). Examples: histplot(), scatterplot(), boxplot().
- **Figure-level functions:** Figure-level functions: Create their own figure (FacetGrid) and manage the entire figure layout. They support faceting via col= and row= parameters. Examples: displot(), catplot(), relplot(), lmplot().

```
# ---------- Axes-level example ----------
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

tips = sns.load_dataset('tips')

# Each function draws on a specific axes
sns.histplot(data=tips, x='total_bill', ax=axes[0])
axes[0].set_title('Distribution of Total Bill')

sns.boxplot(data=tips, x='day', y='total_bill', ax=axes[1])
axes[1].set_title('Bill by Day')

plt.tight_layout()
plt.show()

# ---------- Figure-level example ----------
# displot() creates and manages its own figure
g = sns.displot(data=tips, x='total_bill', col='sex', kde=True)
g.set_titles('Total Bill — {col_name}')  # Use FacetGrid methods
plt.show()
```

Note: *You CANNOT use ax= parameter with figure-level functions. Figure-level functions return a FacetGrid object, not an Axes.*

13.2 Distribution Plots

Distribution plots help you understand the spread and shape of your data. Seaborn provides several functions for this purpose, all of which can be accessed via the unified displot() figure-level wrapper.

histplot() — Bins, KDE Overlay, Hue

histplot() creates a histogram — a bar chart that shows the frequency of values within equal-width intervals (bins). It is the most fundamental distribution visualization tool.

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Load dataset
tips = sns.load_dataset('tips')

# Basic histogram
sns.histplot(data=tips, x='total_bill')
plt.title('Distribution of Total Bill')
plt.show()

# Control number of bins
sns.histplot(data=tips, x='total_bill', bins=20)
plt.show()

# Add KDE (Kernel Density Estimate) overlay
sns.histplot(data=tips, x='total_bill', bins=20, kde=True)
plt.title('Histogram with KDE Overlay')
plt.show()

# Separate by category using hue
sns.histplot(data=tips, x='total_bill', hue='sex', bins=20, kde=True)
plt.title('Total Bill Distribution by Gender')
plt.show()

# Multiple='stack' to stack bars instead of overlapping
sns.histplot(data=tips, x='total_bill', hue='sex',
             multiple='stack', bins=20)
plt.show()
```

Key Parameters: x/y (variable), bins (number or list), kde (bool), hue (grouping variable), multiple ('layer'|'dodge'|'stack'|'fill'), stat ('count'|'frequency'|'density'|'probability')

kdeplot() — Univariate and Bivariate Density

KDE (Kernel Density Estimate) plots show a smooth probability density curve fitted to the data. Unlike histograms, KDE plots are continuous and don't depend on bin width choices.

```
# Univariate KDE — one variable
sns.kdeplot(data=tips, x='total_bill')
plt.title('KDE of Total Bill')
plt.show()

# KDE with shading (fill=True)
sns.kdeplot(data=tips, x='total_bill', fill=True, alpha=0.5)
plt.show()
```

```
# Multiple groups with hue
sns.kdeplot(data=tips, x='total_bill', hue='day', fill=True)
plt.title('Bill Distribution by Day')
plt.show()

# Bivariate KDE — two variables (shows joint density)
sns.kdeplot(data=tips, x='total_bill', y='tip')
plt.title('Joint Density of Bill and Tip')
plt.show()

# Bivariate with filled contours
sns.kdeplot(data=tips, x='total_bill', y='tip',
            fill=True, cmap='Blues')
plt.show()
```

Note: *Bivariate KDE plots reveal where data is most concentrated. High-density areas show the most common combinations of two values.*

ecdfplot() — Empirical Cumulative Distribution

An ECDF (Empirical Cumulative Distribution Function) plot shows what percentage of the data falls below each value. It is extremely useful for comparing distributions across groups.

```
# Basic ECDF
sns.ecdfplot(data=tips, x='total_bill')
plt.title('Cumulative Distribution of Total Bill')
plt.xlabel('Total Bill ($)')
plt.ylabel('Proportion')
plt.show()

# Compare groups — very powerful for quick comparison
sns.ecdfplot(data=tips, x='total_bill', hue='sex')
plt.title('ECDF of Total Bill by Gender')
plt.show()

# Reading the chart: at x=20, the y-value tells you
# what fraction of bills were LESS THAN $20
```

Tip: ECDF is often better than histograms for comparing distributions because it avoids bin-width choices and shows the entire distribution at once.

rugplot() — Marginal Ticks

A rug plot places a small tick mark on the axis for each individual data point. It is rarely used alone but is very effective when layered on top of histograms or KDE plots to show the actual data locations.

```
# Rugplot alone — rarely useful by itself
sns.rugplot(data=tips, x='total_bill')
plt.show()

# Most useful: layered with KDE
sns.kdeplot(data=tips, x='total_bill', fill=True, alpha=0.4)
sns.rugplot(data=tips, x='total_bill', color='red', alpha=0.5)
plt.title('KDE + Rug (showing individual data points)')
plt.show()
```

displot() — Figure-level Wrapper for All Distribution Plots

displot() is the figure-level interface that can produce histograms, KDE plots, and ECDF plots. Its main advantage is built-in faceting using col= and row= parameters.

```
# Histogram using displot
sns.displot(data=tips, x='total_bill', kind='hist', kde=True)
plt.show()

# KDE using displot
sns.displot(data=tips, x='total_bill', kind='kde', fill=True)
plt.show()

# ECDF using displot
sns.displot(data=tips, x='total_bill', kind='ecdf')
plt.show()

# FACETING — split into panels by a category
sns.displot(data=tips, x='total_bill',
            col='sex',          # One column per gender
            row='smoker',       # One row per smoker status
            kind='hist',
            kde=True)
plt.show()
# This creates a 2x2 grid of panels — very powerful!
```

13.3 Categorical Plots ★ (Core Training Topic)

Categorical plots help you compare distributions, central tendencies, and frequencies across groups. This is one of the most commonly used areas of Seaborn in real-world data analysis. We will cover each type in depth.

Boxplot — Understanding the Visual

A boxplot (box-and-whisker plot) summarizes the distribution of a numeric variable for each category. It displays five key statistics: minimum, Q1 (25th percentile), median (50th percentile), Q3 (75th percentile), and maximum (or outlier threshold).

Understanding the elements:

- **Box (IQR):** Box: spans from Q1 to Q3 — the Interquartile Range (IQR). The middle 50% of data lives here.
- **Median Line:** Line inside box: the median (Q2, 50th percentile). Half the data is below this line.
- **Whiskers:** Whiskers: extend to the farthest point within 1.5 × IQR from the box edges.
- **Outliers:** Dots beyond whiskers: individual data points flagged as statistical outliers.

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

# Basic boxplot
sns.boxplot(data=tips, x='day', y='total_bill')
plt.title('Total Bill by Day')
plt.show()

# Add hue for a third dimension
sns.boxplot(data=tips, x='day', y='total_bill', hue='sex')
plt.title('Total Bill by Day and Gender')
plt.legend(title='Gender')
plt.show()

# Horizontal boxplot — useful with many categories
sns.boxplot(data=tips, y='day', x='total_bill')
plt.title('Horizontal Boxplot')
plt.show()

# Customize appearance
sns.boxplot(data=tips, x='day', y='total_bill',
            palette='Set2',       # Color palette
            width=0.5,            # Box width
            flierprops=dict(marker='o', markerfacecolor='red', markersize=5))
plt.show()

# Custom whisker length (default=1.5)
sns.boxplot(data=tips, x='day', y='total_bill', whis=2.0)
# whis=2.0 means whiskers extend to 2.0 * IQR
```

Tip: When boxplots for two groups heavily overlap, the difference is unlikely to be statistically significant. When one group's median is outside the other's IQR box, there is likely a meaningful difference.

Violin Plot — violinplot()

A violin plot combines a box plot with a KDE (density curve). The width of the violin at each point represents the data density at that value. This gives you more information than a boxplot — you can see multimodality (multiple peaks) that a boxplot hides.

```
# Basic violin plot
sns.violinplot(data=tips, x='day', y='total_bill')
plt.title('Violin Plot of Total Bill by Day')
plt.show()

# inner parameter controls what's shown inside the violin
sns.violinplot(data=tips, x='day', y='total_bill',
               inner='box')       # Show box plot inside (default)
sns.violinplot(data=tips, x='day', y='total_bill',
               inner='quartile')  # Show Q1, median, Q3 lines
sns.violinplot(data=tips, x='day', y='total_bill',
               inner='point')     # Show individual data points
sns.violinplot(data=tips, x='day', y='total_bill',
               inner=None)        # Clean violin, no inner marks
plt.show()

# Split violins — perfect for comparing two groups
sns.violinplot(data=tips, x='day', y='total_bill',
               hue='sex', split=True, inner='quartile')
plt.title('Split Violin: Bill by Day and Gender')
plt.show()
# Split=True shows each half of the violin for one group
```

Note: *Use violin plots when you suspect the data has interesting shape — e.g., two distinct groups within a category. If the data is roughly normal, a boxplot may suffice.*

Bar Plot — barplot()

A bar plot shows the estimated central tendency (by default, the mean) of a numeric variable for each category, along with a confidence interval (error bar) that indicates estimation uncertainty.

```
# Basic bar plot — shows mean with 95% CI by default
sns.barplot(data=tips, x='day', y='total_bill')
plt.title('Average Total Bill by Day')
plt.show()

# The error bar (thin line at top) shows 95% confidence interval
# This is NOT the standard deviation — it's the uncertainty of the mean

# Control the error bar type
sns.barplot(data=tips, x='day', y='total_bill',
            errorbar='sd')       # Standard deviation
sns.barplot(data=tips, x='day', y='total_bill',
            errorbar=('ci', 95)) # 95% confidence interval (default)
sns.barplot(data=tips, x='day', y='total_bill',
            errorbar=None)       # No error bar
plt.show()

# Change the estimator function
import numpy as np
sns.barplot(data=tips, x='day', y='total_bill',
            estimator=np.median)  # Show median instead of mean
plt.title('Median Bill by Day')
plt.show()
```

```
# Add hue for grouping
sns.barplot(data=tips, x='day', y='total_bill', hue='sex')
plt.title('Average Bill by Day and Gender')
plt.show()
```

Tip: If you only want to count occurrences, use countplot() instead. barplot() is for aggregating a numeric variable.

Strip Plot — stripplot()

A strip plot shows individual data points for each category. Unlike aggregated plots, you see every single observation. It is very useful for small-to-medium datasets where you want to see the raw data.

```
# Basic strip plot
sns.stripplot(data=tips, x='day', y='total_bill')
plt.title('Individual Bills by Day')
plt.show()

# Add jitter to prevent overplotting
sns.stripplot(data=tips, x='day', y='total_bill',
              jitter=True, alpha=0.6)
plt.show()

# Color by a third variable
sns.stripplot(data=tips, x='day', y='total_bill',
              hue='sex', jitter=True, dodge=True)
# dodge=True separates points by hue group
plt.show()
```

Swarm Plot — swarmplot()

A swarm plot is like a strip plot, but points are arranged so they never overlap. Each point is offset to avoid collisions while keeping the overall position accurate. It provides both individual point visibility and a density impression.

```
# Basic swarm plot
sns.swarmplot(data=tips, x='day', y='total_bill')
plt.title('Swarm Plot of Bills by Day')
plt.show()

# With hue grouping
sns.swarmplot(data=tips, x='day', y='total_bill',
              hue='sex', dodge=True)
plt.show()

# Control point size
sns.swarmplot(data=tips, x='day', y='total_bill', size=4)
plt.show()

# IMPORTANT: Swarm plots can be SLOW with large datasets
# They work best with fewer than ~1000 data points
```

Note: *Swarm plots are slow for large datasets (>1000 rows) because placing points without overlap is computationally expensive. Use strip plots with jitter for large data.*

Count Plot — countplot()

A count plot shows the number of observations in each category. It is essentially a histogram for categorical data.

```
# Basic count plot
sns.countplot(data=tips, x='day')
plt.title('Number of Restaurant Visits by Day')
plt.show()

# Horizontal version
sns.countplot(data=tips, y='day')
plt.show()

# Add hue for a second dimension
sns.countplot(data=tips, x='day', hue='sex')
plt.title('Visit Count by Day and Gender')
plt.show()

# Custom order of categories
sns.countplot(data=tips, x='day',
              order=['Thur', 'Fri', 'Sat', 'Sun'])
plt.show()

# Show count labels on bars
ax = sns.countplot(data=tips, x='day')
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width()/2., p.get_height()),
                ha='center', va='bottom')
plt.show()
```

Point Plot — pointplot()

A point plot shows point estimates (default: mean) with confidence intervals connected by lines across categories. It is ideal for showing trends and interactions between categorical variables.

```
# Basic point plot
sns.pointplot(data=tips, x='day', y='total_bill')
plt.title('Mean Bill by Day with CI')
plt.show()

# With hue — shows interaction between two categorical variables
sns.pointplot(data=tips, x='day', y='total_bill', hue='sex')
plt.title('Mean Bill by Day and Gender')
plt.show()

# The lines connecting points reveal whether the trend is consistent
# across groups. If lines are parallel, no interaction effect.
# If lines cross, there IS an interaction.

# Customize markers and line style
sns.pointplot(data=tips, x='day', y='total_bill', hue='sex',
              markers=['o', 's'],       # Circle and square markers
              linestyles=['-', '--'])   # Solid and dashed lines
plt.show()
```

Combining Plots — Overlaying Plots for Richer Insight

One of the most powerful techniques in Seaborn is overlaying multiple plot types on the same axes. A very common and highly informative combination is placing a strip plot or swarm plot on top of a boxplot or violin plot.

```
# ---- Boxplot + Stripplot ----
fig, ax = plt.subplots(figsize=(10, 6))

# Draw boxplot first (as background)
sns.boxplot(data=tips, x='day', y='total_bill',
            palette='Set2', width=0.5, ax=ax,
            flierprops=dict(marker='', alpha=0))  # Hide outlier dots

# Overlay strip plot with individual points
sns.stripplot(data=tips, x='day', y='total_bill',
              color='black', alpha=0.4, size=4,
              jitter=True, ax=ax)

ax.set_title('Total Bill by Day — Boxplot + Strip')
plt.tight_layout()
plt.show()



# ---- Violinplot + Swarmplot ----
fig, ax = plt.subplots(figsize=(10, 6))

sns.violinplot(data=tips, x='day', y='total_bill',
               inner=None, palette='muted', alpha=0.7, ax=ax)

sns.swarmplot(data=tips, x='day', y='total_bill',
              color='black', alpha=0.6, size=3, ax=ax)

ax.set_title('Violin + Swarm: Full Distribution View')
plt.tight_layout()
plt.show()
```

Tip: The combination of boxplot + stripplot is one of the most commonly used visualizations in data science reports. It shows the summary statistics AND the raw data simultaneously.

catplot() — Figure-level Interface with Faceting

catplot() is the figure-level function that can produce any of the categorical plots above. Its main advantage is that it supports faceting via col= and row= parameters, allowing you to create panel grids.

```
# catplot() with kind= to select plot type
# kind options: 'strip', 'swarm', 'box', 'violin',
#               'boxen', 'point', 'bar', 'count'

# Box plot via catplot
g = sns.catplot(data=tips, x='day', y='total_bill',
                kind='box', height=5, aspect=1.2)
g.set_axis_labels('Day', 'Total Bill ($)')
g.fig.suptitle('Boxplot via catplot()', y=1.02)
plt.show()

# FACETING — split into column panels by a category
g = sns.catplot(data=tips, x='day', y='total_bill',
                kind='box',
```

```
                col='sex',       # Create separate panel per gender
                height=5)
g.set_titles('{col_name}')
plt.show()

# Row AND column faceting
g = sns.catplot(data=tips, x='day', y='total_bill',
                kind='violin',
                col='sex',
                row='smoker',    # Also split by smoker status
                height=4)
plt.show()
# This creates a 2x2 grid: (Male/Female) x (Smoker/Non-smoker)
```

13.4 Regression and Relationship Plots

These plots help you understand the relationships between numeric variables, including correlations, trends, and linear relationships.

scatterplot() — Hue, Size, Style Dimensions

A scatter plot visualizes the relationship between two numeric variables. Seaborn's scatterplot() allows you to encode additional variables using color (hue), point size (size), and point shape (style).

```
tips = sns.load_dataset('tips')

# Basic scatter plot
sns.scatterplot(data=tips, x='total_bill', y='tip')
plt.title('Bill vs Tip')
plt.show()

# Encode a third variable with hue (color)
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='sex')
plt.show()

# Encode a fourth variable with size
sns.scatterplot(data=tips, x='total_bill', y='tip',
                hue='sex', size='size')  # 'size' = table size
plt.show()

# Encode a fifth variable with style (marker shape)
sns.scatterplot(data=tips, x='total_bill', y='tip',
                hue='sex', style='smoker', size='size')
plt.title('Bill vs Tip: Gender, Smoker, Table Size')
plt.show()
# This is ONE plot encoding FOUR variables!
```

lineplot() — With Confidence Intervals and Aggregation

lineplot() connects data points with a line and is often used for time series or ordered data. When multiple observations share the same x value, Seaborn automatically aggregates (default: mean) and adds confidence interval bands.

```
# Load a time-series style dataset
flights = sns.load_dataset('flights')

# Line plot with confidence intervals (automatic aggregation)
sns.lineplot(data=flights, x='year', y='passengers')
plt.title('Average Passengers Over Years')
plt.show()

# Separate lines by month using hue
sns.lineplot(data=flights, x='year', y='passengers', hue='month')
plt.title('Passengers by Month Over Years')
plt.show()

# Control confidence interval level
sns.lineplot(data=flights, x='year', y='passengers',
             errorbar=('ci', 95))     # 95% CI (default)
sns.lineplot(data=flights, x='year', y='passengers',
```

```
             errorbar='sd')           # Standard deviation
sns.lineplot(data=flights, x='year', y='passengers',
             errorbar=None)           # No CI band
plt.show()
```

regplot() — Linear Regression Overlay

regplot() creates a scatter plot with a linear regression line and confidence interval band fitted to the data. It is used to visualize and assess linear relationships.

```
# Basic regression plot
sns.regplot(data=tips, x='total_bill', y='tip')
plt.title('Bill vs Tip with Regression Line')
plt.show()

# Customize appearance
sns.regplot(data=tips, x='total_bill', y='tip',
            scatter_kws={'alpha': 0.5, 'color': 'steelblue'},
            line_kws={'color': 'red', 'linewidth': 2})
plt.show()

# Fit a polynomial regression instead of linear
sns.regplot(data=tips, x='total_bill', y='tip', order=2)
plt.title('Polynomial (degree 2) Regression')
plt.show()

# Suppress confidence interval band
sns.regplot(data=tips, x='total_bill', y='tip', ci=None)
plt.show()
```

Note: *regplot() only handles ONE group at a time. For multiple groups, use lmplot() which supports hue and faceting.*

lmplot() — Faceted Regression Plots

lmplot() is the figure-level version of regplot(). It supports hue (for multiple regression lines on one plot) and col/row for faceting. It creates separate regression fits for each group.

```
# Separate regression lines by sex
sns.lmplot(data=tips, x='total_bill', y='tip', hue='sex')
plt.title('Regression by Gender')
plt.show()

# Facet by column (one panel per smoker status)
sns.lmplot(data=tips, x='total_bill', y='tip', col='smoker')
plt.show()

# Both hue and col
sns.lmplot(data=tips, x='total_bill', y='tip',
           hue='sex', col='smoker', height=5)
plt.show()
```

residplot() — Residual Analysis

A residual plot shows the difference between observed values and values predicted by the regression model. A good linear model should produce residuals that scatter randomly around zero with no pattern.

```
# Residual plot
sns.residplot(data=tips, x='total_bill', y='tip')
plt.axhline(y=0, color='red', linestyle='--')
plt.title('Residuals of Bill vs Tip Regression')
plt.ylabel('Residual')
plt.show()

# Interpretation:
# - Random scatter around zero = linear model is appropriate
# - Clear curve/pattern = non-linear relationship, model needs adjustment
# - Fan shape (spreading residuals) = heteroscedasticity
```

13.5 Matrix and Multivariate Plots

These plots help you understand relationships between multiple variables simultaneously — essential for feature exploration in machine learning and data science.

heatmap() — Annotations, Color Maps, Masking

A heatmap visualizes a matrix of values using color intensity. The most common use is visualizing a correlation matrix — showing how strongly each pair of numeric variables is correlated.

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Load dataset
penguins = sns.load_dataset('penguins').dropna()

# Compute correlation matrix
corr = penguins.select_dtypes(include='number').corr()
print(corr)

# Basic heatmap
sns.heatmap(corr)
plt.title('Correlation Matrix')
plt.show()

# With annotations (show numbers in cells)
sns.heatmap(corr, annot=True, fmt='.2f')
plt.title('Correlation Matrix with Values')
plt.show()

# Choose a diverging colormap (ideal for correlations)
sns.heatmap(corr, annot=True, fmt='.2f',
            cmap='RdBu_r',      # Red for negative, Blue for positive
            vmin=-1, vmax=1)    # Center at 0
plt.show()

# Mask the upper triangle (avoid redundancy)
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, annot=True, fmt='.2f',
            cmap='coolwarm',
            mask=mask,          # Hide upper triangle
            square=True,        # Square cells
            linewidths=0.5)     # Lines between cells
plt.title('Lower Triangle Correlation Heatmap')
plt.tight_layout()
plt.show()
```

Tip: Always use a diverging colormap (RdBu, coolwarm, RdYlGn) for correlation matrices. Center values (near 0) should appear as a neutral color, while extremes (-1, +1) show as contrasting colors.

clustermap() — Hierarchical Clustering with Heatmap

clustermap() creates a heatmap where rows and/or columns are reordered using hierarchical clustering. Similar rows/columns are grouped together, making patterns easier to spot.

```
# Cluster map of correlation matrix
sns.clustermap(corr, annot=True, fmt='.2f',
               cmap='coolwarm', vmin=-1, vmax=1,
               figsize=(8, 8))
plt.title('Clustered Correlation Heatmap')
plt.show()

# Cluster map of raw data (normalize first for fair comparison)
from sklearn.preprocessing import StandardScaler

numeric_cols = penguins.select_dtypes(include='number')
scaled = StandardScaler().fit_transform(numeric_cols)
scaled_df = pd.DataFrame(scaled, columns=numeric_cols.columns)

sns.clustermap(scaled_df.T, cmap='viridis', figsize=(12, 6))
plt.show()
```

pairplot() — Pairwise Relationships Across All Features

pairplot() creates a grid of plots showing every pair of numeric variables. The diagonal shows the distribution of each variable, and the off-diagonal shows scatter plots for each pair. It is one of the fastest ways to get an overview of all relationships in a dataset.

```
penguins = sns.load_dataset('penguins').dropna()

# Basic pairplot — all numeric variables
sns.pairplot(penguins)
plt.show()

# Add hue for class separation — VERY POWERFUL
sns.pairplot(penguins, hue='species')
plt.show()
# Immediately reveals which features separate species best

# Choose specific columns
cols = ['bill_length_mm', 'bill_depth_mm', 'body_mass_g']
sns.pairplot(penguins[cols + ['species']], hue='species')
plt.show()

# Change diagonal plot type
sns.pairplot(penguins, hue='species', diag_kind='kde')
plt.show()

# Change off-diagonal plot type
sns.pairplot(penguins, hue='species', kind='reg')
# kind='reg' adds regression lines to scatter plots
plt.show()
```

Tip: pairplot() with hue='target_variable' is the single best first step for feature exploration in a machine learning project. It immediately shows which features separate classes best.

PairGrid — Custom Diagonal, Upper, Lower Plots

PairGrid gives you complete control over what is plotted in each section of the pair grid. You can assign different plot types to the diagonal, upper triangle, and lower triangle independently.

```
# Custom PairGrid
```

```
g = sns.PairGrid(penguins.dropna(), hue='species')

# Upper triangle: scatter plots
g.map_upper(sns.scatterplot, alpha=0.6)

# Lower triangle: KDE contours
g.map_lower(sns.kdeplot)

# Diagonal: histograms
g.map_diag(sns.histplot, kde=True)

g.add_legend()
plt.show()

# Another useful combination:
g = sns.PairGrid(penguins.dropna())
g.map_upper(sns.regplot)          # Regression in upper triangle
g.map_lower(sns.kdeplot, fill=True)  # Density in lower
g.map_diag(sns.histplot, kde=True)   # Histogram on diagonal
plt.show()
```

13.6 Lab: Statistical Visual Analysis

In this lab you will work through a complete exploratory data analysis using the Titanic, Tips, or Penguins dataset, applying the visualization techniques from this module to build a polished, insight-rich analysis.

Lab Setup and Dataset Loading

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Apply consistent theme
sns.set_theme(style='whitegrid', context='notebook', palette='Set2')

# Load datasets (built into Seaborn)
titanic = sns.load_dataset('titanic')
tips    = sns.load_dataset('tips')
penguins= sns.load_dataset('penguins')

# Quick inspection
print(titanic.head())
print(titanic.info())
print(titanic.describe())
print(titanic.isnull().sum())
```

Task 1: Compare Distributions Across Categories

Use boxplots and violin plots to compare the age and fare distributions across passenger class, sex, and survival status on the Titanic dataset.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Boxplot: Age by Passenger Class
sns.boxplot(data=titanic, x='pclass', y='age',
            hue='survived', palette='Set2', ax=axes[0])
axes[0].set_title('Age Distribution by Class and Survival')
axes[0].set_xlabel('Passenger Class (1=First, 3=Third)')
axes[0].legend(title='Survived', labels=['No', 'Yes'])

# Violin plot: Fare by Sex
sns.violinplot(data=titanic, x='sex', y='fare',
               hue='survived', split=True,
               inner='quartile', palette='muted', ax=axes[1])
axes[1].set_title('Fare Distribution by Sex and Survival')
axes[1].legend(title='Survived', labels=['No', 'Yes'])

plt.suptitle('Titanic: Age and Fare Distributions', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

# ---- Overlay: Boxplot + Stripplot for deeper insight ----
fig, ax = plt.subplots(figsize=(12, 6))
sns.boxplot(data=titanic, x='pclass', y='age',
            palette='pastel', width=0.5, ax=ax,
            flierprops=dict(marker='', alpha=0))
sns.stripplot(data=titanic.dropna(subset=['age']),
```

```
              x='pclass', y='age',
              hue='survived', dodge=True,
              alpha=0.4, size=3, ax=ax)
ax.set_title('Age by Class — Boxplot with Individual Points')
plt.tight_layout()
plt.show()
```

Task 2: Survival Rate Analysis with Categorical Plots

```
fig, axes = plt.subplots(1, 3, figsize=(16, 6))

# 1. Survival count by class
sns.countplot(data=titanic, x='pclass', hue='survived',
              palette='Set2', ax=axes[0])
axes[0].set_title('Survival Count by Class')
axes[0].legend(title='Survived', labels=['No', 'Yes'])

# 2. Mean survival rate by class and sex (bar plot)
sns.barplot(data=titanic, x='pclass', y='survived',
            hue='sex', palette='muted',
            errorbar=('ci', 95), ax=axes[1])
axes[1].set_title('Survival Rate by Class & Sex')
axes[1].set_ylabel('Survival Rate (0-1)')

# 3. Point plot showing interaction
sns.pointplot(data=titanic, x='pclass', y='survived',
              hue='sex', palette='deep',
              markers=['o', 's'],
              linestyles=['-', '--'], ax=axes[2])
axes[2].set_title('Interaction: Class x Sex on Survival')
axes[2].set_ylabel('Survival Rate')

plt.suptitle('Titanic Survival Analysis', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

Task 3: Correlation Heatmap and Pairplot

Use the Penguins dataset to explore feature relationships and correlations.

```
penguins_clean = penguins.dropna()

# ---- Correlation Heatmap ----
fig, ax = plt.subplots(figsize=(8, 6))

numeric_cols = penguins_clean.select_dtypes(include='number')
corr = numeric_cols.corr()

mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr,
            annot=True,
            fmt='.2f',
            cmap='coolwarm',
            vmin=-1, vmax=1,
            mask=mask,
            square=True,
            linewidths=0.5,
            cbar_kws={'shrink': 0.8},
            ax=ax)
ax.set_title('Penguin Features — Correlation Matrix', fontsize=13)
```

```
plt.tight_layout()
plt.show()

# ---- Pairplot ----
g = sns.pairplot(penguins_clean,
                 hue='species',
                 diag_kind='kde',
                 plot_kws={'alpha': 0.6},
                 height=2.5)
g.fig.suptitle('Penguin Pairplot — Feature Relationships by Species',
               y=1.02, fontsize=13)
plt.show()
```

Task 4: Build a Complete Styled Report Figure

Combine multiple plots into a single publication-ready figure with a consistent style.

```
# Apply presentation-ready theme
sns.set_theme(style='whitegrid', context='talk', palette='Set2')

fig = plt.figure(figsize=(18, 12))
fig.suptitle('Penguin Dataset — Exploratory Data Analysis',
             fontsize=18, fontweight='bold', y=0.98)

# ---- Panel 1: Distribution of Bill Length by Species ----
ax1 = fig.add_subplot(2, 3, 1)
sns.boxplot(data=penguins_clean, x='species',
            y='bill_length_mm', palette='Set2', ax=ax1)
sns.stripplot(data=penguins_clean, x='species',
              y='bill_length_mm', color='k',
              alpha=0.3, size=3, jitter=True, ax=ax1)
ax1.set_title('Bill Length by Species')
ax1.set_xlabel('')

# ---- Panel 2: Body Mass Distribution ----
ax2 = fig.add_subplot(2, 3, 2)
sns.violinplot(data=penguins_clean, x='species',
               y='body_mass_g', hue='sex',
               split=True, inner='quartile',
               palette='muted', ax=ax2)
ax2.set_title('Body Mass by Species & Sex')
ax2.set_xlabel('')

# ---- Panel 3: Count by Island ----
ax3 = fig.add_subplot(2, 3, 3)
sns.countplot(data=penguins_clean, x='island',
              hue='species', palette='Set2', ax=ax3)
ax3.set_title('Count by Island & Species')
ax3.set_xlabel('')

# ---- Panel 4: Scatter — Bill dimensions ----
ax4 = fig.add_subplot(2, 3, 4)
sns.scatterplot(data=penguins_clean,
                x='bill_length_mm', y='bill_depth_mm',
                hue='species', style='sex',
                alpha=0.8, ax=ax4)
ax4.set_title('Bill Length vs Bill Depth')

# ---- Panel 5: Flipper Length KDE ----
ax5 = fig.add_subplot(2, 3, 5)
for species, group in penguins_clean.groupby('species'):
```

```
    sns.kdeplot(data=group, x='flipper_length_mm',
                fill=True, alpha=0.4, label=species, ax=ax5)
ax5.set_title('Flipper Length Distribution')
ax5.legend(title='Species')

# ---- Panel 6: Correlation Heatmap ----
ax6 = fig.add_subplot(2, 3, 6)
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, annot=True, fmt='.2f',
            cmap='coolwarm', vmin=-1, vmax=1,
            mask=mask, square=True,
            linewidths=0.5, ax=ax6)
ax6.set_title('Feature Correlations')

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig('penguin_eda_report.png', dpi=150,
            bbox_inches='tight', facecolor='white')
plt.show()
print('Figure saved as penguin_eda_report.png')
```

Quick Reference: Seaborn Function Cheat Sheet

Function	Type	Purpose	Returns
histplot()	Axes-level	Histogram with optional KDE	Axes
kdeplot()	Axes-level	Kernel density estimate	Axes
ecdfplot()	Axes-level	Cumulative distribution	Axes
displot()	Figure-level	Wrapper for dist plots	FacetGrid
boxplot()	Axes-level	Box and whisker plot	Axes
violinplot()	Axes-level	Violin + KDE plot	Axes
barplot()	Axes-level	Bar chart with CI	Axes
stripplot()	Axes-level	Individual point scatter	Axes
swarmplot()	Axes-level	Non-overlapping points	Axes
countplot()	Axes-level	Count by category	Axes
pointplot()	Axes-level	Connected point estimates	Axes
catplot()	Figure-level	Wrapper for cat plots	FacetGrid
scatterplot()	Axes-level	Scatter with hue/size	Axes
lineplot()	Axes-level	Line with CI bands	Axes
regplot()	Axes-level	Scatter + regression	Axes
lmplot()	Figure-level	Faceted regression	FacetGrid
residplot()	Axes-level	Regression residuals	Axes
heatmap()	Axes-level	Matrix visualization	Axes
clustermap()	Figure-level	Clustered heatmap	ClusterGrid
pairplot()	Figure-level	Pairwise relationships	PairGrid

Tip: Always call sns.set_theme() at the start of your script to ensure consistent styling. Use figure-level functions (catplot, displot, relplot) when you need faceting across subgroups.

Note: *Seaborn 0.12+ renamed the ci= parameter to errorbar=. If you see deprecation warnings about ci, replace ci=95 with errorbar=('ci', 95).*

Vikas Karanth U at 2/22/2026 3:20 PM